

Министерство науки и высшего образования Российской Федерации
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ**
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
Университет ИТМО

Факультет прикладной информатики

Образовательная программа «Мобильные и сетевые технологии»

Лабораторная работа 5

По дисциплине «Проектирование и реализация баз данных»

Процедуры функции и триггеры в PostgreSQL

Вариант 11 БД Автомастерская

Выполнил: Харьковский Семён

Группа: КЗ341

Преподаватель: Говорова М.М.

Дата: 21.09.2026

Санкт-Петербург 2026

1 Цель работы

Овладеть практическими навыками создания и использования хранимых процедур, функций и триггеров в базе данных PostgreSQL, а также освоить выполнение и проверку разработанных объектов в консоли SQL Shell psql.

2 Практическое задание

- Создать три хранимые процедуры согласно заданию 4 индивидуального варианта 11.
- Создать семь необходимых триггеров для базы данных «Автомастерская».
- Проверить процедуры и триггеры на корректных и некорректных данных в psql.
- Представить код объектов, вызовы, результаты выполнения и выводы.

Процедуры варианта должны повышать цены деталей Ford на 10 процентов, повышать разряд мастеров, отремонтировавших более трёх автомобилей, и определять число автомобилей, отремонтированных каждым мастером за истекший квартал.

3 Выполнение работы

3.1 Исходная база данных

Работа выполняется в базе autoservice_v11, схема autoservice, созданной в лабораторной работе 3 и дополненной контрольными данными лабораторной работы 4. Для реализации первой процедуры добавлена таблица part_brand_compatibility, связывающая детали с марками автомобилей. Для хранения истории цен создана таблица work_price_audit.

3.1.1 Логическая модель

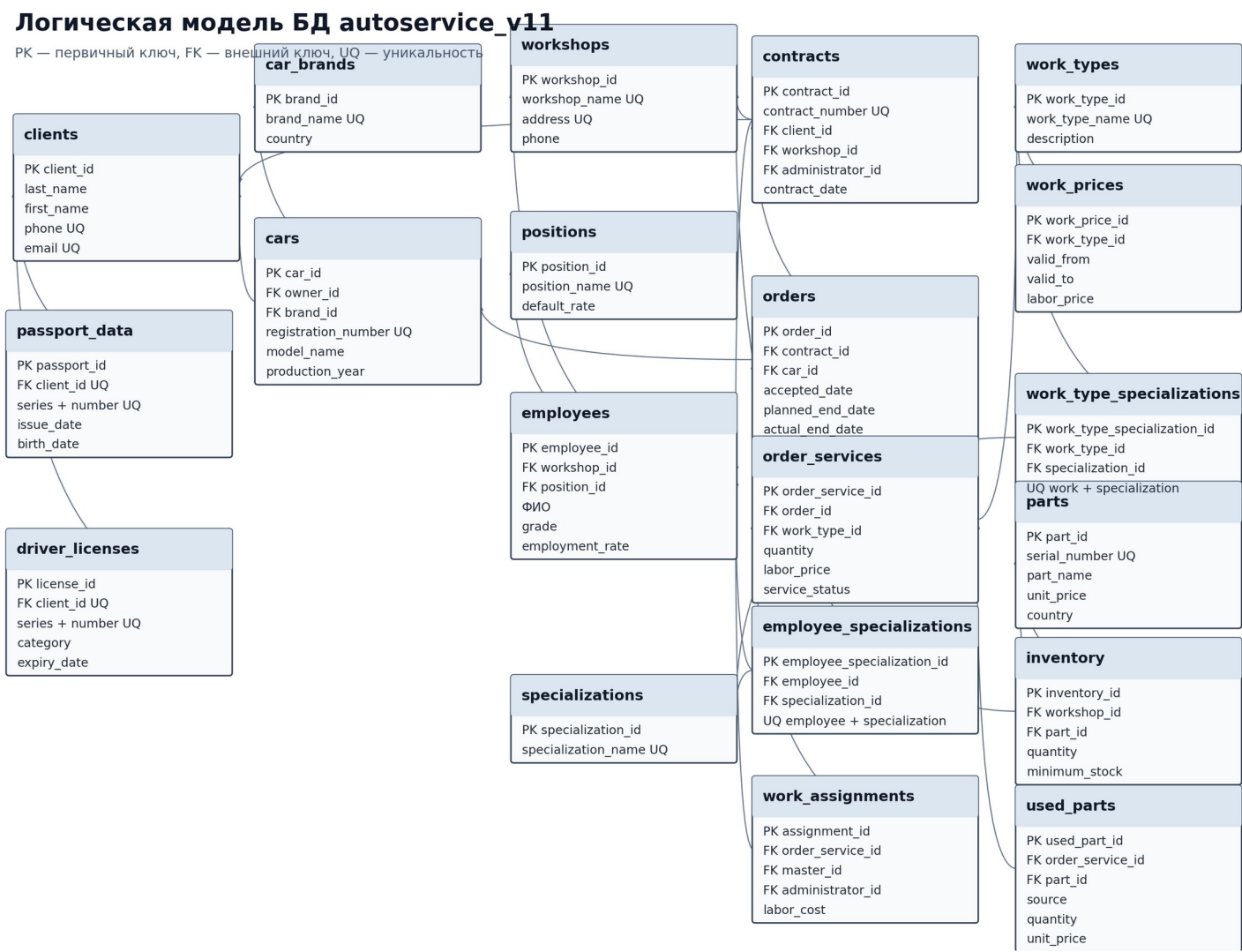


Рисунок 1 — Логическая модель исходной базы данных autoservice_v11

3.2 Хранимые процедуры

Процедуры написаны на языке PL/pgSQL и вызываются оператором CALL. Для возврата количества изменённых строк применены параметры INOUT и GET DIAGNOSTICS. Третья процедура возвращает набор строк через курсор.

3.2.1 Повышение цены деталей Ford

Процедура получает название марки и процент повышения. Перед обновлением проверяется допустимость процента. Детали выбираются через таблицу совместимости, поэтому изменение не зависит от того, использовалась ли деталь в предыдущих заказах.

```
CREATE OR REPLACE PROCEDURE p_increase_part_prices(
    IN p_brand_name TEXT DEFAULT 'Ford',
    IN p_percent NUMERIC DEFAULT 10,
    INOUT p_updated_rows INTEGER DEFAULT 0)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_percent <= 0 OR p_percent > 100 THEN
        RAISE EXCEPTION 'Процент повышения должен быть в диапазоне (0; 100]';
    END IF;
    UPDATE parts p
    SET unit_price = ROUND(p.unit_price * (1 + p_percent / 100.0), 2)
    WHERE EXISTS (
        SELECT 1 FROM part_brand_compatibility pbc
        JOIN car_brands b ON b.brand_id = pbc.brand_id
        WHERE pbc.part_id = p.part_id
        AND LOWER(b.brand_name) = LOWER(p_brand_name));
    GET DIAGNOSTICS p_updated_rows = ROW_COUNT;
END;
$;
```

Деталь	Цена до, руб.	Цена после, руб.
Масляный фильтр	950,00	1 045,00
Втулка стабилизатора	1 200,00	1 320,00

Таблица 1 — Контрольный результат CALL p_increase_part_prices()

3.2.2 Повышение разряда продуктивных мастеров

Процедура считает завершённые заказы каждого мастера. Один заказ соответствует ремонту одного автомобиля. Мастер с количеством ремонтов больше заданного получает следующий разряд, но не выше шестого.

```
CREATE OR REPLACE PROCEDURE p_promote_productive_masters(
    IN p_min_repaired_cars INTEGER DEFAULT 3,
    INOUT p_promoted_rows INTEGER DEFAULT 0)
LANGUAGE plpgsql AS $$
BEGIN
    WITH productive_masters AS (
        SELECT wa.master_id
        FROM work_assignments wa
        JOIN order_services os ON os.order_service_id = wa.order_service_id
        JOIN orders o ON o.order_id = os.order_id
        WHERE o.actual_end_date IS NOT NULL
        GROUP BY wa.master_id
        HAVING COUNT(DISTINCT o.order_id) > p_min_repaired_cars)
    UPDATE employees e
    SET grade = LEAST(e.grade + 1, 6)
    FROM productive_masters pm
    WHERE e.employee_id = pm.master_id AND e.grade < 6;
    GET DIAGNOSTICS p_promoted_rows = ROW_COUNT;
END;
$;
```

Мастер	Завершённых ремонтов	Разряд до	Разряд после
Кузнецов	4	5	6

Таблица 2 — Контрольный результат CALL p_promote_productive_masters()

3.2.3 Количество ремонтов за квартал

Начало квартала передаётся входным параметром. Процедура открывает курсор с фамилией, именем мастера и количеством завершённых заказов в трёхмесячном интервале. LEFT JOIN сохраняет в результате мастеров без завершённых ремонтов.

```
CREATE OR REPLACE PROCEDURE p_repaired_cars_by_master(
  IN p_quarter_start DATE,
  INOUT p_result REFCURSOR DEFAULT 'quarter_repair_counts')
LANGUAGE plpgsql AS $$
BEGIN
  OPEN p_result FOR
    SELECT e.employee_id, e.last_name, e.first_name,
           COUNT(DISTINCT o.order_id) AS repaired_car_count
    FROM employees e
    JOIN positions pos ON pos.position_id = e.position_id
    LEFT JOIN work_assignments wa ON wa.master_id = e.employee_id
    LEFT JOIN order_services os ON os.order_service_id = wa.order_service_id
    LEFT JOIN orders o ON o.order_id = os.order_id
    AND o.actual_end_date >= p_quarter_start
    AND o.actual_end_date < p_quarter_start + INTERVAL '3 months'
    WHERE pos.position_name = 'мастер'
    GROUP BY e.employee_id, e.last_name, e.first_name
    ORDER BY repaired_car_count DESC;
END;
$$;

BEGIN;
CALL p_repaired_cars_by_master(
  DATE_TRUNC('quarter', CURRENT_DATE)::DATE, 'quarter_repair_counts');
FETCH ALL FROM quarter_repair_counts;
COMMIT;
```

Мастер	Отремонтировано автомобилей
Кузнецов	4
Волков	1
Морозов	1

Таблица 3 — Контрольный результат процедуры за текущий квартал

3.3 Триггеры

Разработано семь триггеров уровня строки. BEFORE-триггеры проверяют или корректируют данные до записи, AFTER-триггеры синхронизируют связанные таблицы и формируют аудит после успешной операции.

№	Триггер	Событие	Назначение
1	trg_assignment_master_qualification	BEFORE INSERT/UPDATE	Проверяет специализацию мастера
2	trg_assignment_completion_sync	AFTER UPDATE	Завершает услугу и заказ
3	trg_used_part_consume_inventory	BEFORE INSERT	Проверяет и уменьшает остаток
4	trg_used_part_restore_inventory	AFTER DELETE	Возвращает деталь на склад
5	trg_work_price_audit	AFTER INSERT/UPDATE/DELETE	Записывает историю цен
6	trg_protect_issued_order	BEFORE UPDATE	Защищает выданный заказ
7	trg_manage_work_price_period	BEFORE INSERT	Закрывает прежний период цены

3.3.1 Проверка специализации мастера

Триггер сравнивает набор специализаций мастера с требованиями выбранного вида работы. Если отсутствует хотя бы одна обязательная специализация, операция отклоняется исключением.

```
CREATE FUNCTION fn_check_master_qualification() RETURNS TRIGGER AS $$
DECLARE v_work_type_id INTEGER;
BEGIN
    SELECT work_type_id INTO v_work_type_id FROM order_services
    WHERE order_service_id = NEW.order_service_id;
    IF EXISTS (SELECT 1 FROM work_type_specializations
               WHERE work_type_id = v_work_type_id)
        AND EXISTS (
            SELECT 1 FROM work_type_specializations wts
            WHERE wts.work_type_id = v_work_type_id
            AND NOT EXISTS (
                SELECT 1 FROM employee_specializations es
                WHERE es.employee_id = NEW.master_id
                AND es.specialization_id = wts.specialization_id)) THEN
        RAISE EXCEPTION 'Мастер не имеет требуемой специализации';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_assignment_master_qualification
BEFORE INSERT OR UPDATE OF master_id, order_service_id ON work_assignments
FOR EACH ROW EXECUTE FUNCTION fn_check_master_qualification();
```

Проверка	Результат psql
Назначить Кузнецова на замену тормозных колодок	ERROR: мастер не имеет требуемой специализации

3.3.2 Автоматическое завершение услуги и заказа

После заполнения completed_at триггер проверяет все назначения услуги. Если они завершены, услуга получает статус «выполнена». Когда выполнены все услуги заказа, заказ получает статус «готов» и фактическую дату окончания.

```
CREATE FUNCTION fn_sync_service_and_order_completion() RETURNS TRIGGER AS $$
DECLARE v_order_id INTEGER;
BEGIN
    IF NEW.completed_at IS NOT NULL
        AND OLD.completed_at IS DISTINCT FROM NEW.completed_at THEN
```

```

IF NOT EXISTS (SELECT 1 FROM work_assignments
WHERE order_service_id = NEW.order_service_id AND completed_at IS NULL) THEN
UPDATE order_services SET service_status = 'выполнена'
WHERE order_service_id = NEW.order_service_id;
END IF;
SELECT order_id INTO v_order_id FROM order_services
WHERE order_service_id = NEW.order_service_id;
IF NOT EXISTS (SELECT 1 FROM order_services
WHERE order_id = v_order_id AND service_status <> 'выполнена') THEN
UPDATE orders SET actual_end_date = COALESCE(actual_end_date, CURRENT_DATE),
status = 'готов'
WHERE order_id = v_order_id;
END IF;
END IF;
RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_assignment_completion_sync AFTER UPDATE OF completed_at
ON work_assignments FOR EACH ROW
EXECUTE FUNCTION fn_sync_service_and_order_completion();

```

Состояние	Статус услуги	Статус заказа	Фактическая дата
До	выполняется	ремонт	NULL
После	выполнена	готов	CURRENT_DATE

3.3.3 Списание детали со склада

Перед добавлением детали источника «автомастерская» определяется мастерская заказа. Строка inventory блокируется FOR UPDATE, проверяется наличие запаса, после чего количество уменьшается.

```

CREATE FUNCTION fn_consume_inventory_part() RETURNS TRIGGER AS $$
DECLARE v_workshop_id INTEGER; v_stock INTEGER;
BEGIN
IF NEW.source = 'автомастерская' THEN
SELECT ct.workshop_id INTO v_workshop_id
FROM order_services os JOIN orders o ON o.order_id = os.order_id
JOIN contracts ct ON ct.contract_id = o.contract_id
WHERE os.order_service_id = NEW.order_service_id;
SELECT quantity INTO v_stock FROM inventory
WHERE workshop_id = v_workshop_id AND part_id = NEW.part_id FOR UPDATE;
IF NOT FOUND OR v_stock < NEW.quantity THEN
RAISE EXCEPTION 'Недостаточный складской остаток';
END IF;
UPDATE inventory SET quantity = quantity - NEW.quantity
WHERE workshop_id = v_workshop_id AND part_id = NEW.part_id;
END IF;
RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_used_part_consume_inventory BEFORE INSERT ON used_parts
FOR EACH ROW EXECUTE FUNCTION fn_consume_inventory_part();

```

Деталь	Остаток до	Списано	Остаток после
Масляный фильтр	12	1	11

3.3.4 Возврат детали на склад

После удаления записи used_parts количество детали возвращается на склад мастерской заказа. OLD используется, поскольку удалённая строка недоступна через NEW.

```

CREATE FUNCTION fn_restore_inventory_part() RETURNS TRIGGER AS $$
DECLARE v_workshop_id INTEGER;
BEGIN

```

```

IF OLD.source = 'автомастерская' THEN
  SELECT ct.workshop_id INTO v_workshop_id
  FROM order_services os JOIN orders o ON o.order_id = os.order_id
  JOIN contracts ct ON ct.contract_id = o.contract_id
  WHERE os.order_service_id = OLD.order_service_id;
  UPDATE inventory SET quantity = quantity + OLD.quantity
  WHERE workshop_id = v_workshop_id AND part_id = OLD.part_id;
END IF;
RETURN OLD;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_used_part_restore_inventory AFTER DELETE ON used_parts
FOR EACH ROW EXECUTE FUNCTION fn_restore_inventory_part();

```

Деталь	Остаток до	Возвращено	Остаток после
Масляный фильтр	12	1	13

3.3.5 Аудит стоимости работ

AFTER-триггер обрабатывает INSERT, UPDATE и DELETE одной функцией. TG_OP определяет операцию, OLD содержит прежнее состояние, NEW — новое. В журнале сохраняются цены, периоды, пользователь и время изменения.

```

CREATE FUNCTION fn_audit_work_price() RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO work_price_audit(work_price_id, work_type_id, operation,
      new_labor_price, new_valid_from, new_valid_to)
    VALUES(NEW.work_price_id, NEW.work_type_id, TG_OP,
      NEW.labor_price, NEW.valid_from, NEW.valid_to); RETURN NEW;
  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO work_price_audit(work_price_id, work_type_id, operation,
      old_labor_price, new_labor_price, old_valid_from, old_valid_to,
      new_valid_from, new_valid_to)
    VALUES(NEW.work_price_id, NEW.work_type_id, TG_OP,
      OLD.labor_price, NEW.labor_price, OLD.valid_from, OLD.valid_to,
      NEW.valid_from, NEW.valid_to); RETURN NEW;
  ELSE
    INSERT INTO work_price_audit(work_price_id, work_type_id, operation,
      old_labor_price, old_valid_from, old_valid_to)
    VALUES(OLD.work_price_id, OLD.work_type_id, TG_OP,
      OLD.labor_price, OLD.valid_from, OLD.valid_to); RETURN OLD;
  END IF;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_work_price_audit AFTER INSERT OR UPDATE OR DELETE
ON work_prices FOR EACH ROW EXECUTE FUNCTION fn_audit_work_price();

```

Операция	Цена до	Цена после	Назначение записи
UPDATE	2 500,00	2 500,00	Закрытие прежнего периода
INSERT	NULL	2 750,00	Новая стоимость

3.3.6 Защита выданного заказа

Заказ со статусом «выдан» считается закрытым документом. BEFORE UPDATE сравнивает OLD и NEW и отклоняет любое изменение такой строки.

```

CREATE FUNCTION fn_protect_issued_order() RETURNS TRIGGER AS $$
BEGIN
  IF OLD.status = 'выдан' AND NEW IS DISTINCT FROM OLD THEN
    RAISE EXCEPTION 'Заказ % уже выдан и не может быть изменён', OLD.order_id;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_protect_issued_order BEFORE UPDATE ON orders

```

```
FOR EACH ROW EXECUTE FUNCTION fn_protect_issued_order();
```

Проверка	Результат psql
Изменить planned_end_date заказа 1	ERROR: заказ 1 уже выдан и не может быть изменён

3.3.7 Управление периодами стоимости

При добавлении новой цены триггер закрывает предыдущий открытый период днём перед valid_from новой записи. Затем пересечения проверяются операторами диапазонов daterange.

```
CREATE FUNCTION fn_manage_work_price_period() RETURNS TRIGGER AS $$
BEGIN
    UPDATE work_prices SET valid_to = NEW.valid_from - 1
    WHERE work_type_id = NEW.work_type_id
    AND valid_to IS NULL AND valid_from < NEW.valid_from;
    IF EXISTS (
        SELECT 1 FROM work_prices wp
        WHERE wp.work_type_id = NEW.work_type_id
        AND DATERANGE(wp.valid_from, COALESCE(wp.valid_to, 'infinity'), '[]')
        && DATERANGE(NEW.valid_from, COALESCE(NEW.valid_to, 'infinity'), '[]')) THEN
        RAISE EXCEPTION 'Период новой цены пересекается с существующим';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_manage_work_price_period BEFORE INSERT ON work_prices
FOR EACH ROW EXECUTE FUNCTION fn_manage_work_price_period();
```

Запись	valid_from	valid_to	Цена, руб.
Предыдущая	01.01.2026	CURRENT_DATE	2 500,00
Новая	CURRENT_DATE + 1	NULL	2 750,00

3.4 Проверка в SQL Shell psql

Сценарий запускается командой psql -U postgres -d autoservice_v11 -f. Метакоманда ON_ERROR_STOP прекращает выполнение при непредусмотренной ошибке. Ожидаемые ошибки триггеров 1 и 6 перехватываются во вложенных блоках EXCEPTION и выводятся как NOTICE. Изменяющие проверки заключены в BEGIN и ROLLBACK.

- CALL вызывает процедуру; SELECT использовать для процедуры нельзя.
- Для чтения refcursor требуется открытая транзакция: BEGIN, CALL, FETCH ALL, COMMIT.
- RAISE EXCEPTION отменяет текущий оператор и предотвращает нарушение бизнес-правила.
- ROLLBACK возвращает цены, разряды, остатки и статусы в исходное состояние.

4 Вывод

В ходе лабораторной работы разработаны три хранимые процедуры индивидуального варианта 11 и семь триггеров для базы данных автомастерской.

Процедуры выполняют массовое изменение цен, повышение разряда мастеров и формирование квартальной статистики. Триггеры проверяют квалификацию исполнителя, синхронизируют завершение услуг и заказов, поддерживают складские остатки, ведут аудит цен, защищают закрытые заказы и исключают пересечение периодов стоимости. Применение параметров IN и INOUT, курсора, OLD, NEW, TG_OP, BEFORE и AFTER обеспечивает выполнение бизнес-правил на стороне PostgreSQL независимо от клиентского приложения.

Список источников

1. PostgreSQL Documentation. CREATE PROCEDURE. URL: <https://www.postgresql.org/docs/current/sql-createprocedure.html> (дата обращения: 21.09.2026).
2. PostgreSQL Documentation. PL/pgSQL Trigger Functions. URL: <https://www.postgresql.org/docs/current/plpgsql-trigger.html> (дата обращения: 21.09.2026).
3. PostgreSQL Documentation. psql. URL: <https://www.postgresql.org/docs/current/app-psql.html> (дата обращения: 21.09.2026).
4. Методические указания к лабораторной работе 5 «Процедуры, функции и триггеры в PostgreSQL».

Приложение А Состав SQL сценария

Полный исполняемый листинг приложен отдельным файлом Харьковский_ЛР5_Вариант11.sql. Сценарий предназначен для psql и состоит из следующих разделов:

Раздел	Содержание
0	Таблица совместимости деталей, марка Ford и таблица аудита
1	Три хранимые процедуры варианта 11
2	Семь триггерных функций и семь триггеров
3	Контрольные вызовы процедур
4	Проверки корректных и ошибочных операций триггеров
5	Проверка объектов в системном каталоге PostgreSQL